Laboratório Funcional · Aula 12

Vitória Souza Santos · RA 24229424-2 · Turma ESOFT-C
Gerado em 19/10/2026, 15:02:03 · tempo registrado no cronômetro: 24:26

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

A função agora() é impura porque depende do horário atual, então pode retornar resultados diferentes em chamadas realizadas em momentos diferentes. Já a função registra(msg) é impura porque modifica o estado externo da execução ao exibir uma mensagem na tela.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: 1: A-1 A-2
Linha 4: 4: 14
Linha 7: 7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função modifica a variável global contador a cada chamada. Por isso, a mesma entrada produz resultados diferentes. Isso dificulta os testes porque o resultado depende do estado anterior do programa, tornando os testes menos previsíveis.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

- compor(f, g)(4): primeiro calcula g(4) = 12, depois f(12) = 14.
- compor(g, f)(4): primeiro calcula f(4) = 6, depois g(6) = 18.

Os resultados são diferentes porque a ordem de aplicação das funções muda.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) devolve uma nova lista ordenada e deixa notas intacta. notas.sort() altera a própria lista e devolve None. sorted(notas) segue melhor o estilo funcional, pois evita mutação e permite trabalhar com um novo valor.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo é a abreviação de quote: impede que a expressão seguinte seja avaliada como chamada de função e a trata como dado literal. Sem ele, (A B C) é interpretado como aplicação da função A aos argumentos B e C; como A não está definido, ocorre erro de identificador/variável não definido.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
Expressão: (car (cdr (cdr '(A B C))))
Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 3: comum em 6 passos, até 2 multiplicações esperando (expressão de até 24 caracteres); em cauda em 5 passos, nada esperando (até 16 caracteres).
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 5: comum em 10 passos, até 4 multiplicações esperando (expressão de até 36 caracteres); em cauda em 7 passos, nada esperando (até 18 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Em Scheme, espero que funcione porque conta-ate está em recursão de cauda e o Scheme otimiza esse padrão, reutilizando o quadro da pilha. Em Python, 500 chamadas deve funcionar, mas um milhão deve ultrapassar o limite de recursão e gerar RecursionError.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
A primeira chamada imprime 500. A segunda termina com RecursionError: maximum recursion depth
exceeded (a mensagem pode variar ligeiramente conforme a versão do Python).
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum cresce a pilha de chamadas e ficam pendentes as multiplicações: cada chamada precisa esperar o resultado da próxima para multiplicar ao voltar. Na versão em cauda, o resultado parcial vai no acumulador e a chamada recursiva é a última operação; não fica trabalho pendente e o Scheme pode reutilizar o quadro da pilha.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme implementa otimização de chamada em cauda, obrigatória no padrão Scheme: como a chamada é a última operação, o quadro pode ser reutilizado, mantendo espaço constante. Python não faz essa otimização; suas chamadas acumulam quadros e o limite padrão de recursão é baixo (aproximadamente mil), então a versão com um milhão de chamadas lança RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (if (null? lis)
      '()
      (if (even? (car lis))
          (cons (car lis) (pares (cdr lis)))
          (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais delicada foi construir a lista de resultado em pares sem inverter a ordem. Em todas as funções, null? reconhece o caso-base da lista vazia e cdr avança para o restante. tamanho soma 1 por cada car; soma-quadrados usa car para elevar o primeiro número ao quadrado; pares testa o car com even? e usa cons para incluí-lo no resultado, ou apenas recursa no cdr se for ímpar.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares deve imprimir [0,1,4,9,16]. primeiro 42 (div 1 0) deve imprimir 42, porque primeiro não usa o segundo argumento.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

Ao habilitar member 17 squares, a busca não termina: percorre a lista infinita sem encontrar
17.
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

A avaliação preguiçosa calcula a lista infinita squares apenas até o trecho pedido por take 5, portanto não tenta materializar a lista toda. Haskell também só avalia um argumento quando seu valor é necessário; primeiro devolve o primeiro argumento e ignora div 1 0, então a divisão por zero nunca é calculada.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 17 squares não termina porque squares contém 0, 1, 4, 9, 16, 25, …; 17 não aparece e a busca continua para sempre. member 16 squares termina ao encontrar 16. Como a lista é crescente, a busca pode parar quando o elemento atual ultrapassar o procurado: member b (a:x) | b == a = True | b < a = False | otherwise = member b x.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15. [1, 2, 3].map(vezes(5)) imprime [5, 10, 15].

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[Cabo, Monitor, Mouse, Teclado]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

somador(10) devolve uma closure (fechamento) que mantém uma referência ao ambiente léxico em que foi criada, incluindo o valor n=10. O valor continua acessível enquanto a função devolvida existir, mesmo depois de somador terminar.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

compose aplica primeiro a função recebida e depois a função que chamou compose: f.compose(g).apply(4)=f(g(4))=14. andThen faz f primeiro e g depois: f.andThen(g).apply(4)=g(f(4))=18. A expressão equivalente a compor(f,g), definida como f(g(x)), é f.compose(g).apply(4).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
Exercício 1 (Java):
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
Exercício 2 (JavaScript):
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];
const aprovados = alunos
  .filter(aluno ⇒ aluno.nota ⩾ 6)
  .map(aluno ⇒ aluno.nome.toUpperCase());
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
Exercício 1 (Java): 244
Exercício 2 (JavaScript): ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Para estes exemplos, a versão funcional é mais fácil de ler porque expressa diretamente a transformação (filtrar e mapear), sem expor o controle do laço e a variável acumuladora. Para depurar passo a passo ou inspecionar valores intermediários, o laço costuma ser mais simples; em um pipeline, posso dividir as etapas em variáveis ou inserir inspeções.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

Para este cálculo, acho a versão funcional mais fácil de entender: ela mostra a sequência gerar números → elevar ao cubo → somar, enquanto a versão imperativa detalha índice e acumulador. Aceitaria o custo aproximado em relatórios, processamento em lote ou serviços sem prazo rígido, onde clareza e facilidade de teste importam. Não o aceitaria sem medição e otimização em sistemas de tempo real, embarcados ou de alta frequência com limites estritos de latência.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O mais surpreendente foi que Haskell consegue trabalhar com listas infinitas sem calculá-las por inteiro, avaliando apenas o trecho realmente necessário.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro é “b <- h” na compreensão da primeira partição: <- é um gerador que exige uma lista, mas h é o pivô (um elemento). Além disso, h aparece como elemento individual em [h] e na comparação b > h, gerando tipos incompatíveis. Corrige-se para “b <- t, b <= h”: b vem da cauda t e b <= h é o filtro. A mensagem do compilador indica que inferiu h como uma lista para usá-lo como gerador, mas o restante do código exige que h tenha o tipo de um elemento; a correção é fazê-los concordar.